Comparing Fairness Interventions in GNNs Across Two Benchmarks

Ananya Batra
Master of Engineering
University of California, Irvine
Irvine, CA
ananyb2@uci.edu

Zhijie Liu
Master of Enigineering
University of California, Irvine
Irvine, CA
zhijiel9@uci.edu

I-Ling Lai
Master of Engineering
University of California, Irvine
Irvine, CA
ilingl@uci.edu

I. Introduction

Graph Neural Networks (GNNs) have become a standard approach for node classification tasks because they leverage both node features and relational structures. But GNNs are particularly prone to 2 sources of bias. Feature bias arises when node features correlate with sensitive attributes, while structural bias arises since nodes tend to connect to others with similar sensitive characteristics. Through message passing, GNNs aggregate information from neighboring nodes, which can propagate and reinforce patterns correlated with sensitive attributes across the graph. As a result, biases present in node features or graph structure may be amplified during representation learning, ultimately leading to unfair predictions. Therefore, in real applications we need to consider both accuracy and fairness. When looking into different fairness methods for GNNs, we observed that they can be roughly grouped into three categories: preprocessing (modifying the graph structure, such as deleting edges), representation learning (learning embeddings that remove sensitive information, such as fair contrastive learning), and in-model methods (adding fairness objectives during training, such as adversarial debiasing). We are interested in comparing these 3 different techniques over different datasets and experimenting with whether chaining different techniques leads to improved results. To test this hypothesis, our implementations will include:

1) a baseline GCN,
2) FairDrop + GCN,
3) fair contrastive pre-training + GCN,
4) FairGNN and
5) a full pipeline combined FairDrop + fair contrastive learning + FairGNN

to test whether these approaches are complementary.

II. Background and Literature Review

Recent work has shown that fairness issues in graph learning are fundamentally different from i.i.d. settings because edges encode social/behavioral relations and message passing can amplify existing group disparities. Surveys and literature reviews in fairness on relational/graph data highlight how bias can arise from correlated node features, homophily with respect to sensitive attributes, and training objectives that do not explicitly control group-level disparities. [4] In node classification, group fairness is often operationalized through metrics such as statistical parity difference and equal opportunity difference, defined as differences in positive prediction rates across sensitive groups and differences in true positive rates across groups conditioned on positive ground truth, respectively. [2] A key line of work addresses graph fairness at the model level. FairGNN is a representative approach that integrates a GNN classifier with an adversarial component to discourage sensitive attribute information from being encoded in node embeddings, aiming to reduce downstream discrimination while preserving predictive utility. [1] In addition to model-based debiasing, recent methods intervene before or alongside supervision. FairDrop proposes a preprocessing strategy that deletes edges deemed "biased"… aiming to reduce bias propagation through message passing. [5] Meanwhile, fairness-aware graph contrastive learning explores self-supervised representation learning where augmentations (e.g., feature masking or edge deletion) are designed specifically to reduce intrinsic bias while maintaining task-relevant information. [3] Motivated by these complementary mechanisms—structure-level (FairDrop), representation-level (fair contrastive pre-training), and training-level (FairGNN)—our project evaluates them side-by-side and in combination to understand when they are synergistic versus redundant.

III. Problem Statement and Research Questions

Problem. In fairness-sensitive node classification, GNNs can achieve good accuracy while still showing large group disparities, because message passing can propagate and amplify feature and structural bias.

Primary research question. Across Pokec and NBA, which intervention stage provides the best fairness–utility trade-off—preprocessing (FairDrop), representation-level debiasing (fair contrastive pretraining), or in-model adversarial debiasing (FairGNN)—and does stacking them produce additional gains?

Optional questions. How robust are results to dataset properties (e.g., homophily, feature–sensitive correlation), and do stacked interventions create diminishing returns or optimization conflicts?

IV. Datasets, Tasks, and Metrics

Datasets. We use two commonly adopted fairness-in-graphs benchmarks: **Pokec** and **NBA**.

Task + sensitive attribute (Pokec). Occupation prediction with sensitive attribute **gender**.

Metrics. We evaluate accuracy plus two group-fairness gaps:

- Δ_{SP}: difference in predicted positive rates across sensitive groups (statistical parity difference).
- Δ_{EO}: difference in TPR/FPR (or TPR gap in standard definitions) across groups (equal opportunity difference).

(Implementation note: we will report $\Delta_{\mathrm{SP}}/\Delta_{\mathrm{EO}}$ exactly as computed in our evaluation code to match baseline outputs.)

V. Proposed Methods

We compare three representative fairness paradigms that intervene at distinct stages and therefore target different bias mechanisms: preprocessing, representation learning, and in-model objectives.

Pipelines to implement (all under consistent splits / training budgets):

1) **Baseline GCN** (no fairness)
2) **FairDrop + GCN** (preprocessing)
3) **Fair Contrastive Pretraining + GCN** (representation-level, self-supervised)
4) **FairGNN** (in-model adversarial debiasing)
5) **Stacked pipeline**: FairDrop $\rightarrow$ fair contrastive $\rightarrow$ FairGNN

VI. Key Theory

A. Why bias can be amplified by GNN message passing

As mentioned previously, GNNs aggregate information from neighboring nodes, which improves prediction but can also propagate existing biases. When node features correlate with sensitive attributes or the graph exhibits homophily (nodes with similar attributes are more likely to be connected), repeated message passing can reinforce biased patterns in the learned embeddings, leading to unfair predictions. In particular, GCN-style aggregation mixes information primarily within densely connected groups, causing representations to increasingly encode sensitive-group signals. A typical GCN-style update is:

$$H^{(l+1)} = \sigma\Big(\tilde{D}^{-\frac{1}{2}}\tilde{A}\tilde{D}^{-\frac{1}{2}}H^{(l)}W^{(l)}\Big)$$

where $\tilde{A}$ is the adjacency matrix with self-loops, $\tilde{D}$ is its degree matrix, and σ is a nonlinearity. When edges are disproportionately within the same sensitive group, repeated multiplication by $\tilde{A}$ tends to mix information mostly within the group. This can make embeddings increasingly encode group membership and group-linked signals.

B. Fairness metrics used in our evaluation

We evaluate both utility and group fairness. We use Accuracy for utility and two group fairness gaps for fairness [2]:

1) Statistical Parity Difference (Δ_{SP}): difference in predicted positive rates across sensitive groups.

$$\Delta_{\mathrm{SP}} = |P(\hat{y}=1 \mid S=0) - P(\hat{y}=1 \mid S=1)|$$

2) Equal Opportunity Difference (Δ_{EO}): difference in model performance on positive ground-truth instances across groups (often measured by TPR gap; in some settings also reported with related rates).

$$\Delta_{\mathrm{EO}} = |\mathrm{TPR}(S=0) - \mathrm{TPR}(S=1)|$$

Our goal is not to optimize fairness alone, but to study the fairness–accuracy trade-off across datasets and intervention stages.

C. Representation-level intervention: fairness-aware contrastive learning

We first learn node representations in a self-supervised way, before downstream node classification. Contrastive learning constructs two "views" of the same graph via augmentations, encodes both views with a shared encoder, and enforces that the embeddings of the same node from two views are close, while embeddings of different nodes are separated [3].

We generate two views:

$$\tilde{G}^{(1)} = \Big(\tilde{A}^{(1)}, \tilde{X}^{(1)}\Big), \quad \tilde{G}^{(2)} = \Big(\tilde{A}^{(2)}, \tilde{X}^{(2)}\Big)$$

Encode them with the same GNN encoder $f()$:

$$H^{(1)} = f\Big(\tilde{A}^{(1)}, \tilde{X}^{(1)}\Big), \quad H^{(2)} = f\Big(\tilde{A}^{(2)}, \tilde{X}^{(2)}\Big)$$

Positive and negative pairs. For each node i, $(h_i^{(1)}, h_i^{(2)})$ forms a positive pair, while $(h_i^{(1)}, h_j^{(2)})$ with $j \neq i$ are treated as negative pairs.

Standard random feature masking / edge deletion treats all features and edges equally. If some features or edge patterns are strong proxies for sensitive attributes, random corruption does not specifically suppress these biased signals. As a result, embeddings can still encode sensitive information, and fairness may not improve [3]. In contrast, the above approach motivates "fairness-aware" augmentations that target biased features and biased structural patterns more aggressively.

1) Fairness-aware feature masking (targets feature bias): We mask features that are strongly correlated with the sensitive attribute more frequently, so the encoder cannot rely on them.

Step 1: correlation test and p-values. For each feature i, compute its correlation with the sensitive attribute S and obtain a p-value $p_i^{(c)}$. A smaller p-value indicates stronger correlation with S and thus a higher risk of being a sensitive proxy.

Step 2: convert correlation signal to keep probability. Let $p^{(b)}$ be a base masking rate (hyperparameter). Define $p_i^{(f)}$ as the probability of keeping feature i. A simple monotonic mapping consistent with our draft is:

$$p_i^{(f)} = \left(1 - p^{(b)}\right) \cdot p_i^{(c)}$$

This makes strongly correlated features (small $p_i^{(c)}$) have smaller keep probabilities, so they are masked more often.

Step 3: (sampling + apply). Sample a mask with Bernoulli variables and apply it by element-wise multiplication (Hadamard product), as in your "Equation 4" step:

$$m = \left[\mathrm{Bernoulli}(p_1^{(f)}), \ldots, \mathrm{Bernoulli}(p_F^{(f)})\right], \quad \tilde{X} = X \odot m$$

This directly enforces the principle: if a feature leaks sensitive information, the model is forced to learn alternatives.

2) Fairness-aware edge deletion (targets structural bias): To reduce structural bias, we create augmented graph views where the edge patterns encode sensitive attributes less strongly. There are many different deletion strategies but we focus on triangle-based edge deletion: edges inside same-group triangles are deleted more aggressively. The motivation is that triangles form dense "echo chambers." In message passing, such redundant within-group connectivity can repeatedly reinforce group-specific signals. By weakening these structures in augmented views, the contrastive objective encourages embeddings that are less dependent on sensitive-group connectivity patterns.

D. Preprocessing intervention: FairDrop (biased edge dropout)

FairDrop modifies the input graph before training by dropping edges that are likely to amplify biased propagation [5]. Conceptually, it aims to reduce the degree to which neighborhood aggregation can transmit sensitive-group information. This intervention is model-agnostic and can be applied to standard GCN pipelines. The main trade-off is that removing edges can also remove useful task-relevant information, so we must evaluate fairness gains against potential accuracy loss.

E. In-model intervention: FairGNN (adversarial debiasing)

FairGNN addresses fairness during supervised training by discouraging sensitive information from being encoded in node embeddings [1]. A common formulation is an adversarial objective, where a predictor is trained for the label task, while an adversary tries to predict the sensitive attribute from embeddings. The GNN is trained to make this sensitive prediction difficult:

$$\min_{\mathrm{GNN}} \ \mathcal{L}_{\mathrm{task}}(Y, \hat{Y}) - \lambda \max_{\mathrm{adv}} \ \mathcal{L}_{\mathrm{adv}}(S, \hat{S}(H))$$

Here, L_{task} encourages accurate label prediction, while the adversarial term pushes embeddings H to be uninformative about S. This directly targets sensitive leakage. The practical trade-offs include hyperparameter sensitivity (λ) and optimization stability, which can vary across datasets.

F. Why stacking interventions may help (and why it may not)

Our project compares and also stacks interventions across three stages:

1) Preprocessing (FairDrop): changes A to reduce biased propagation paths.
2) Representation learning (fair contrastive pretraining): shapes embeddings through fairness-aware views ($\tilde{X}$, $\tilde{A}$).
3) In-model (FairGNN): explicitly suppresses sensitive information during supervised learning.

Because these interventions target different bias channels, stacking them could be complementary. For example, FairDrop can weaken structural amplification, fair contrastive learning can reduce reliance on sensitive proxies during pretraining, and FairGNN can enforce invariance during label supervision. However, while stacking interventions may reduce bias more effectively, combining these many approaches might lower accuracy or make training unstable. Our main question is therefore which stage offers the best fairness–accuracy balance, and whether combining methods actually provides additional benefit across datasets.

G. Preliminary Results (Completed Milestone: Baseline on both datasets)

We have already implemented the **baseline GCN** and computed **Accuracy / Δ_{SP} / Δ_{EO}** on both datasets.

```
Epoch 020 | Loss 0.4891 | Train 0.756 | Val 0.687 | Test 0.693 | ΔSP 0.462 | ΔEO 0.642
Epoch 040 | Loss 0.3506 | Train 0.839 | Val 0.672 | Test 0.691 | ΔSP 0.526 | ΔEO 0.698
Epoch 060 | Loss 0.2260 | Train 0.896 | Val 0.656 | Test 0.683 | ΔSP 0.599 | ΔEO 0.758
Epoch 080 | Loss 0.1439 | Train 0.944 | Val 0.659 | Test 0.675 | ΔSP 0.504 | ΔEO 0.663
Epoch 100 | Loss 0.1961 | Train 0.905 | Val 0.674 | Test 0.687 | ΔSP 0.495 | ΔEO 0.667
Epoch 120 | Loss 0.1545 | Train 0.947 | Val 0.665 | Test 0.686 | ΔSP 0.544 | ΔEO 0.710
Epoch 140 | Loss 0.0962 | Train 0.973 | Val 0.657 | Test 0.676 | ΔSP 0.534 | ΔEO 0.693
Epoch 160 | Loss 0.0607 | Train 0.989 | Val 0.651 | Test 0.670 | ΔSP 0.535 | ΔEO 0.688
Epoch 180 | Loss 0.0389 | Train 0.997 | Val 0.652 | Test 0.667 | ΔSP 0.537 | ΔEO 0.686
Epoch 200 | Loss 0.1076 | Train 0.902 | Val 0.628 | Test 0.634 | ΔSP 0.633 | ΔEO 0.744
```

Fig. 1. Pokec (baseline GCN)

Pokec (baseline GCN): test accuracy is around 0.69 early in training, while $\Delta_{\mathrm{SP}}/\Delta_{\mathrm{OE}}$ remain large (e.g., Epoch 20: Test 0.693, Δ_{SP} 0.462, $/\Delta_{\mathrm{OE}}$ 0.642; Epoch 60: Test 0.683, Δ_{SP} 0.599, $/\Delta_{\mathrm{OE}}$ 0.758). This indicates strong fairness issues even when accuracy is reasonable.

```
Epoch 020 | Loss 0.3892 | Train 0.856 | Val 0.694 | Test 0.672 | ΔSP 0.158 | ΔEO 0.167
Epoch 040 | Loss 0.2486 | Train 0.925 | Val 0.694 | Test 0.672 | ΔSP 0.201 | ΔEO 0.208
Epoch 060 | Loss 0.1067 | Train 0.989 | Val 0.694 | Test 0.609 | ΔSP 0.154 | ΔEO 0.000
Epoch 080 | Loss 0.0258 | Train 1.000 | Val 0.694 | Test 0.562 | ΔSP 0.091 | ΔEO 0.000
Epoch 100 | Loss 0.0120 | Train 1.000 | Val 0.677 | Test 0.578 | ΔSP 0.089 | ΔEO 0.042
Epoch 120 | Loss 0.0098 | Train 1.000 | Val 0.677 | Test 0.562 | ΔSP 0.043 | ΔEO 0.042
Epoch 140 | Loss 0.0097 | Train 1.000 | Val 0.694 | Test 0.562 | ΔSP 0.043 | ΔEO 0.042
Epoch 160 | Loss 0.0097 | Train 1.000 | Val 0.694 | Test 0.562 | ΔSP 0.043 | ΔEO 0.042
Epoch 180 | Loss 0.0178 | Train 0.989 | Val 0.661 | Test 0.500 | ΔSP 0.074 | ΔEO 0.125
Epoch 200 | Loss 0.0775 | Train 0.947 | Val 0.710 | Test 0.531 | ΔSP 0.234 | ΔEO 0.167
```

Fig. 2. NBA (baseline GCN)

NBA (baseline GCN): early epochs reach Test $\approx$ 0.67 with smaller fairness gaps (e.g., Epoch 20: Test 0.672, Δ_{SP} 0.158, $/\Delta_{\mathrm{OE}}$ 0.167; Epoch 40: Test 0.672, Δ_{SP} 0.201, $/\Delta_{\mathrm{OE}}$ 0.208), but later training shows test degradation, suggesting the importance of consistent validation/early stopping.

(We will include these baselines as the "reference point" for all debiasing methods under the same data splits.)

H. Experimental Plan and Evaluation Protocol

- Use fixed train/val/test masks per dataset.
- For each method, tune a small set of hyperparameters on
- validation (including fairness–utility trade-off).
- Report mean ± std over multiple random seeds.
- Output: fairness–accuracy trade-off plots, plus an ablation on the stacked pipeline.

I. Expected Deliverables and Success Criteria

We will produce controlled comparisons and a taxonomy table summarizing: intervention stage, bias source targeted, mechanism, fairness gain, accuracy impact, and stability/complexity. Success criteria include fairness improvement vs baseline, controlled accuracy degradation, and reproducibility.

References

[1] Enyan Dai and Suhang Wang. Say no to the discrimination: Learning fair graph neural networks with limited sensitive attribute information. In *Proceedings of the Web Conference (WWW)*, 2021.

[2] Yushun Dong et al. A benchmark for fairness-aware graph learning. *arXiv preprint arXiv:2407.12112*, 2024.

[3] Öykü Deniz Köse and Yanning Shen. Fairness-aware node representation learning. *arXiv preprint arXiv:2106.05391*, 2021.

[4] Charlotte Laclau, Christine Largeron, and Manvi Choudhary. A survey on fairness for machine learning on graphs. *arXiv preprint arXiv:2205.05396*, 2024.

[5] Indro Spinelli et al. Fairdrop: Biased edge dropout for enhancing fairness in graph representation learning. *IEEE Transactions on Artificial Intelligence*, 2022.